EXPERIMENT NO: 06

Aim: Implement an Election Algorithm

Theory:

Election algorithm and distributed processing

Distributed algorithm:

Distributed algorithm runs on a distributed system. Distributed system is a collection of independent computers that do not share their memory. Each processor has its own memory and they communicate via communication networks. Communication in networks is implemented in a process on one machine communicating with a process on another machine. Many algorithms used in the distributed system require a coordinator that performs functions needed by other processes in the system.

Election algorithms are designed to choose a coordinator.

Election Algorithms: Election algorithms choose a process from a group of processors to act as a coordinator. If the coordinator process crashes due to some reasons, then a new coordinator is elected on other processor. Election algorithm basically determines where a new copy of the coordinator should be restarted. Election algorithm assumes that every active process in the system has a unique priority number. The process with highest priority will be chosen as a new coordinator. Hence, when a coordinator fails, this algorithm elects that active process which has highest priority number. Then this number is send to every active process in the distributed system. We have two election algorithms for two different configurations of a distributed system.

1. The Bully Algorithm – This algorithm applies to system where every process can send a message to every other process in the system. Algorithm – Suppose process P sends a message to the coordinator.

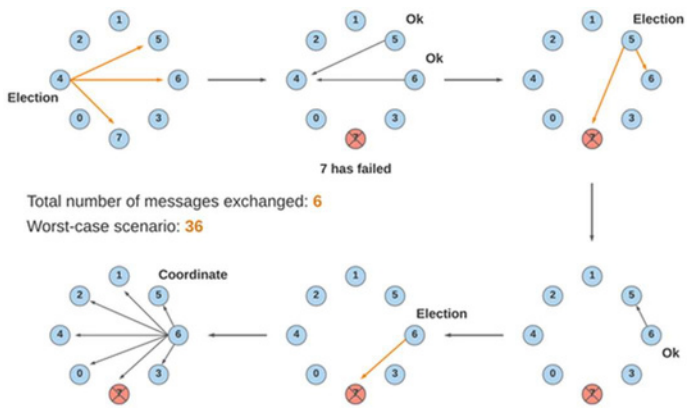
1. If the coordinator does not respond to it within a time interval T, then it is assumed that coordinator has failed.
2. Now process P sends an election messages to every process with high priority number.
3. It waits for responses, if no one responds for time interval T then process P elects itself as a coordinator.
4. Then it sends a message to all lower priority number processes that it is elected as their new coordinator.
5. However, if an answer is received within time T from any other process Q,
   - (I) Process P again waits for time interval T’ to receive another message from Q that it has been elected as coordinator.
   - (II) If Q doesn’t respond within time interval T’ then it is assumed to have failed and algorithm is restarted.

2. The Ring Algorithm – This algorithm applies to systems organized as a ring(logically or physically). In this algorithm we assume that the link between the process are unidirectional and every process can message to the process on its right only. Data structure that this algorithm uses is active list, a list that has a priority number of all active processes in the system.

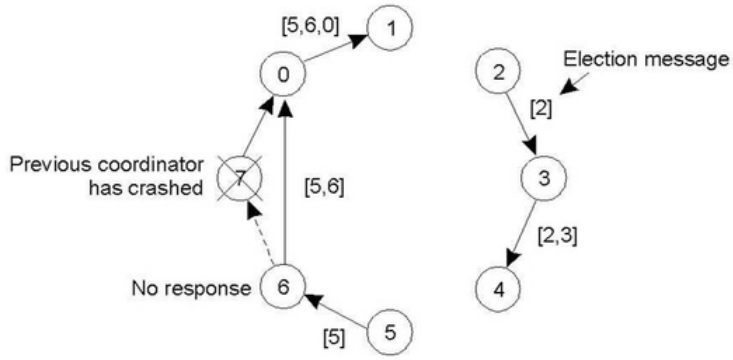
Algorithm –

1. If process P1 detects a coordinator failure, it creates new active list which is empty initially.
2. If process P2 receives message elect from processes on left, it responds in 3 ways:
   - (I) If message received does not contain 1 in active list then P1 adds 2 to its active list and forwards the message.
   - (II) If this is the first election message it has received or sent, P1 creates new active list with numbers 1 and 2. It then sends election message 1 followed by 2.
   - (III) If Process P1 receives its own election message 1 then active list for P1 now contains numbers of all the active processes in the system. Now Process P1 detects highest priority number from list and elects it as the new coordinator.

Bully Algorithm



A Ring Algorithm



```
class Pro:
    def __init__(self, id):
        self.id = id
        self.act = True  # Flag to check if the process is active

class GFG:
    def __init__(self):
        self.TotalProcess = 0
        self.process = []

    def initialiseGFG(self):
        print("Number of processes: 5")
        self.TotalProcess = 5
        self.process = [Pro(i) for i in range(self.TotalProcess)]

    def Election(self):
        # Simulating a failure of a process
        print(f"Process no {self.process[self.FetchMaximum()].id} fails")
        self.process[self.FetchMaximum()].act = False
        print("Election Initiated by Process 2")
        initializedProcess = 2

        old = initializedProcess
        newer = (old + 1) % self.TotalProcess  # Start passing election to next process

        while True:
            if self.process[newer].act:
                print(f"Process {self.process[old].id} passes Election({self.process[old].id}) to Process {self.process[newer].id}")
                old = newer  # Move old to newer
            newer = (newer + 1) % self.TotalProcess
            if newer == initializedProcess:  # End when we come back to the initiating process
                print(f"Process {self.process[old].id} passes Election({self.process[old].id}) to Process {self.process[initializedProcess].id}")
                break  # Exit after passing back to the initiating process

        # Declaring the coordinator (leader)
        print(f"Process {self.process[self.FetchMaximum()].id} becomes coordinator")
        coord = self.process[self.FetchMaximum()].id

        old = coord
        newer = (old + 1) % self.TotalProcess

        # Sending the coordinator message
        while True:
            if self.process[newer].act:
                print(f"Process {self.process[old].id} passes Coordinator({coord}) message to Process {self.process[newer].id}")
                old = newer  # Move old to newer
            newer = (newer + 1) % self.TotalProcess
            if newer == coord:  # End when coordinator receives its own message back
                print("End of Election")
                break

    def FetchMaximum(self):
        # Fetch the process with the maximum ID that is still active
        maxId = -9999
        ind = 0
        for i in range(self.TotalProcess):
            if self.process[i].act and self.process[i].id > maxId:
                maxId = self.process[i].id
                ind = i
        return ind

def main():
    object = GFG()
    object.initialiseGFG()
    object.Election()

if __name__ == "__main__":
    main()
```

```
Number of processes: 5
Process no 4 fails
Election Initiated by Process 2
Process 2 passes Election(2) to Process 3
Process 3 passes Election(3) to Process 0
Process 0 passes Election(0) to Process 1
Process 1 passes Election(1) to Process 2
Process 3 becomes coordinator
Process 3 passes Coordinator(3) message to Process 0
Process 0 passes Coordinator(3) message to Process 1
Process 1 passes Coordinator(3) message to Process 2
End of Election
```

Conclusion: The experiment successfully implemented a leader election algorithm, where processes in a ring elect the highest ID process as coordinator, ensuring fault tolerance and efficient message passing.

```
class Pro:
    def __init__(self, id):
        self.id = id
        self.act = True  # Flag to check if the process is active

class GFG:
    def __init__(self):
        self.TotalProcess = 0
        self.process = []

    def initialiseGFG(self):
        print("Number of processes: 5")
        self.TotalProcess = 5
        # Create 5 processes with IDs 0 through 4
        self.process = [Pro(i) for i in range(self.TotalProcess)]

    def Election(self):
        # 1. Simulating failure of the highest ID process (the current coordinator)
        max_idx = self.FetchMaximum()
        print(f"Process no {self.process[max_idx].id} fails")
        self.process[max_idx].act = False

        # 2. Start Election
        print("Election Initiated by Process 2")
        initializedProcess = 2
        old = initializedProcess
        newer = (old + 1) % self.TotalProcess

        while True:
            if self.process[newer].act:
                print(f"Process {self.process[old].id} passes Election message to Process {self.process[newer].id}")
                old = newer

            newer = (newer + 1) % self.TotalProcess

            # End when we come back to the initiating process
            if newer == (initializedProcess + 1) % self.TotalProcess:
                break

        # 3. Declaring the coordinator (leader)
        new_coord_idx = self.FetchMaximum()
        coord_id = self.process[new_coord_idx].id
        print(f"\nProcess {coord_id} becomes coordinator")

        # 4. Sending the coordinator message around the ring
        old = new_coord_idx
        newer = (old + 1) % self.TotalProcess

        while True:
            if self.process[newer].act:
                print(f"Process {self.process[old].id} passes Coordinator({coord_id}) message to Process {self.process[newer].id}")
                old = newer

            if newer == new_coord_idx:
                print("End of Election")
                break

            newer = (newer + 1) % self.TotalProcess

    def FetchMaximum(self):
        """Finds the index of the active process with the highest ID."""
        maxId = -1
        ind = -1
        for i in range(self.TotalProcess):
            if self.process[i].act and self.process[i].id > maxId:
                maxId = self.process[i].id
                ind = i
        return ind

def main():
    obj = GFG()
    obj.initialiseGFG()
    obj.Election()

if __name__ == "__main__":
    main()
```